A Mostly Harmless Introduction to Claude Code, Codex and Friends

For Empirical Researchers

Daniel Sánchez Pazmiño

September 3, 2026

About Me

Daniel Sánchez, MA

- Economist, MA in Economics, Simon Fraser University (Canada)
- Economic Statistician, Government of Alberta (Canada)
- Co-director, Ecuadorian Development Research Lab (LIDE)

Working with agentic AI

- Design GenAI workflows (Claude, ChatGPT, Gemini, Copilot) for economic research/analysis
- Developing Model Context Protocol servers for data access with agentic AI assistants
- Training researchers and analysts on agentic AI for different purposes

Agenda

1. Introduction: what AI is, and why agentic AI
2. What's out there: tools and models
3. Before we get started: cybersecurity basics
4. Usage
   - vibe coding & running regressions
   - code review & file organization
   - help your agent help you
   - literature review
   - writing review & SKILL.md
   - skills from other researchers
   - MCP for data access
5. Cybersecurity: local models

AI, Anyone?

- What people call "AI" today mostly means one thing: large language models (LLMs), not the general concept of "AI"
  - technically, we've had AI for a while (ML, even regression)
  - but the current wave of AI is powered by LLMs
- An LLM is trained to predict the next word from huge amounts of text and code
  - hence the "token" talk: tokens are groups of words
- Three things had to come together for that to work:
  - Data: decades of text and code, digitized and sitting on the internet
  - Compute: GPUs cheap and parallel enough to train on data at that scale
  - Architecture: the Transformer (2017), which made training at that scale practical

A Word on Tokens

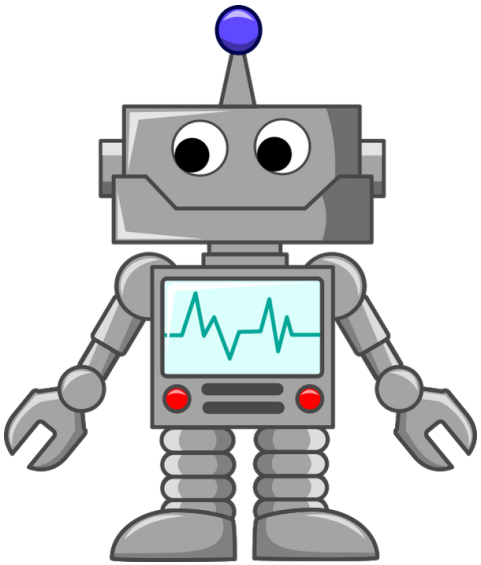
- Usage is metered in *tokens*: chunks of the text a model reads and writes
- Every prompt and every response consumes tokens
  - Big tasks (long documents, large codebases) consume more
- Subscriptions (e.g. Claude Pro/Max) come with usage caps; the API instead charges per token, pay only for what you use

Usage Caps: the Cake Analogy

- Think of a subscription's weekly allowance as a whole cake baked for the week
  - Every prompt and every response cuts a slice
  - Run out early, and there's no cake left until it resets
- Two caps, not one:
  - A **5-hour rolling window**: a small cake that resets every 5 hours
  - A **weekly cap**: a big cake that has to last the whole week
  - Hitting the 5-hour cap just pauses you; hitting the weekly cap means no more cake until next week (or topping up via the API)

Vanilla AI vs. Agentic AI

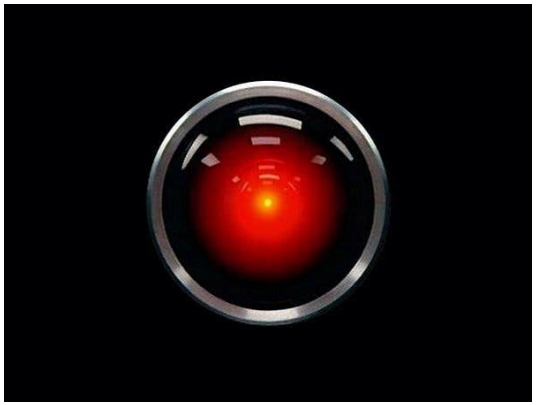
Vanilla AI (ChatGPT, Claude.ai, a chat window)



- You paste in code or data, it replies with text
- You copy the suggestion back out, apply it yourself, rerun it yourself
- It never sees whether its own suggestion actually worked
- We've had it for a while, but at a certain point, it stops being *that* useful

Vanilla AI vs. Agentic AI (cont.)

Agentic AI (Claude Code, Codex, and friends)



- It gained *agency*: acts on its own, not just replying to you
- It opens your files, makes the edit itself, runs the script itself
- It reads the real error or output and tries again, without you relaying it
- The unit of delegation moves from *a question* to *a task*
- What made this possible: larger context windows and reliable tool use, not simply "a smarter model"

Why even care?

- Point the AI to your paper folder: the sky's the limit.
- Read your raw data, write and run the cleaning script, and notice when a value label silently changed across survey waves
- It can run your regression, read the actual error message, and fix the bug, instead of you pasting the traceback back and forth

What it does **not** do

- It does **not** replace your judgment on identification, specification, or interpretation: you still decide what's correct, it does the retyping and rerunning
  - Many times, they hallucinate a solution that looks plausible but is actually wrong
  - No context or economic intuition: it's a *stochastic parrot*, it cannot think
- Net effect: fewer hours lost to mechanical iteration [writing every code line by line], more time on the parts of the research that actually require you
- It doesn't know what cybersecurity or data privacy rules apply to your data, so you still have to be careful

What's out there

- Terminal / CLI agents: Claude Code (Anthropic), Codex (Open AI), Cursor (now SpaceX), GitHub Copilot (Microsoft), etc.
  - Each of these has its own desktop program version and/or IDE plugins.
- Non-technical agentic software:
  - ChatGPT Work
  - Claude Cowork
  - Copilot Cowork

And what about the “models”?

- AI bros increasingly talk about “models”
- In practice, the model is the “engine” that powers the agent
- Anthropic: Haiku (fast), Sonnet (default), Opus (powerful)
- OpenAI: Luna (fast), Terra (default), Sol (powerful)
- Gemini: Gemini fast, Gemini pro
- Grok, Kimi, Deepseek, so many more...

Before we get started

- Cybersecurity is a real concern:
  - Secret/private data with sensitive info (i.e. social security numbers) shouldn't be used into an agentic AI
  - For this, there's *local* models, which we may discuss later,
- Dont' paste stuff from webpages you don't trust into the agent, it may be malicious
  - Prompt injection: trick your AI into doing something you didn't intend
- An agent can delete your whole project folder if you tell it to
  - Generally they have a "classifier" which tells them whether a command is safe or not, but it can be tricked
  - Use Git!

Usages: Vibe coding a data cleaning script

- Vibe coding: coined in 2025, means prompting the agent to write code while you watch and comment
  - The agent writes the code
  - You provide the vibes
- Case study: 12 files of employment survey data from Ecuador
  - We need to clean, select the right variables, and reshape it into a province-month panel.

Usages: Running some regressions

- The agent can run the regression for you, and help you interpret the output
- You can even use it for multiple specifications
  - Maybe you didn't have time before to code so many of them, now you do it in less than 10 mins

Usages: Code Review and File Organization

- One of the most painful things ever is reviewing a coauthor's code or some old code you wrote a year ago
  - The folders are a mess
  - The comments are terrible
  - You never wrote documentation, lol
- An agent can help you out by
  - Reviewing the code and suggesting improvements
  - Suggesting a better folder structure (and implementing it)
  - Writing documentation for your code for you (or for another agent)
- Case study: my own replication package for Holcombe & Boudreaux (2015), *Regulation and Corruption*, and analyze

Holcombe, R. G., & Boudreaux, C. J. (2015). Regulation and corruption. *Public Choice*, 164, 75-85.

Usages: help your agent help you

- Agents love markdown format instructions
  - Markdown:
    - Use headings to separate tasks
    - Use bullet points to list instructions
    - Use code blocks for code snippets
- You can ask your own AI to see your messy code/project, and write an AGENTS.md file
  - Tells what the agent should do, and how to do it
  - With Claude Code, AGENTS.md needs to be called CLAUDE.md
- Write READMEs, replication manuals, etc. etc. etc.

Usages: Literature Review

- Review an existing literature review
- Look for similar papers, and summarize them
  - best done using the "deep research" capability of agents
- Use connectors/MCPs to investigate specific literature databases

Usages: Writing Review and SKILL.md

- Arguable the worst ability of LLMs is writing a good paper
- However, one can create a reproducible skill to teach the LLM to write like yourself
- This is a skill:
  - A reusable prompt that can be used to write a paper in your style
  - Can be shared with other researchers, and they can adapt it to their own style
- Ask the agent to look at your old papers, draft a SKILI and learn it.

Usages: Skills and Agents from Modern researchers

- A few economists have already started to create skills and agents for their own research
- Paul Goldsmith-Pinkham (Yale SOM): writes and teaches on this directly
  - Runs "Using AI in Research and Teaching," a course at Yale SOM
  - Substack series on building and using SKILL.md skills for research
- Scott Cunningham (Baylor, *Causal Inference: The Mixtape*): publishes actual skills, not just posts about them
  - MixtapeTools: a public repo of skills (coding audits, deck-building, bib-checking, and more)
- Pedro H.C. Sant'Anna (Emory, of Callaway-Sant'Anna DiD fame): a ready-to-fork Claude Code template for academic LaTeX/Beamer + R projects
  - Multi-agent review, quality gates, replication protocols built in

Usages: MCP for Data Access

- Ever had ChatGPT complain because it can't get you a specific datapoint?
  - Either it's private, or it's not easily reachable by an agent
- MCPs are add-ons that allow agents to access specific data sources, like APIs or databases
  - I.e. MCPs to access government data, or to access specific academic databases (i.e. JSTOR, Scopus, etc.)
  - Or MCPs to allow the agent to work more easily with other agents, (stata-mcp)
- EcuDataMCP: allows to easily consult the Ecuadorian governemnt database
- Similar MCPs available for US, Canada, etc.

Cybersecurity: Local models

- Increasingly, researchers are thinking of ways on how to leverage the power of agents with restricted data
- Private datasets (i.e. tax records) contain sensitive information, and cannot be shared with external agents
- For this reason, open source models (Kimi, Llama) can potentially be installed and deployed in a local (powerful) computer
  - Most models aren't open source (i.e. Claude, ChatGPT, Gemini, Copilot), and require sending your data to the cloud
  - Open source vs. open weights: true "open source" require the weights to be available, not just the code
  - Open source models are not as powerful, and depend highly on your hardware capabilities
  - Kimi is the strongest these days, but can change quickly

References

- Goldsmith-Pinkham, P. *Skills: Specifying How an Agent Should Think*. A Causal Affair. paulgp.substack.com/p/skills-specifying-how-an-agent-should
- Cunningham, S. *MixtapeTools*: tools for coding, teaching, and presentations with AI assistance. github.com/scunning1975/MixtapeTools
- Sant'Anna, P. H. C. *claude-code-my-workflow*: a ready-to-fork Claude Code template for academics using LaTeX/Beamer + R. github.com/pedrohcgs/claude-code-my-workflow